



Hyena Hierarchy

Towards Larger Convolutional Language Models

Bài báo: Poli, Massaroli, Nguyen, Dao, Baccus, Bengio, Ermon, Ré · Stanford & Mila · ICML 2023 (PMLR 202)

Nhóm 08 · CS2308

Trần Tú Quang · Tô Huỳnh Minh Tiến · Nguyễn Cao Trung Kiên

GVHD: TS. Nguyễn Văn Kiệt · University of Information Technology, VNU-HCM (UIT) · 2026



1. **Bối cảnh & động lực** – vì sao cần thay thế attention
2. **Nhắc lại nền tảng** – Language Modeling, Perplexity, Transformer
3. **Self-Attention & nút thắt** – cơ chế Q, K, V và chi phí $O(L^2)$
4. **Khoảng cách năng lực & Related Work** – SSM $\rightarrow$ H3/GSS $\rightarrow$ Hyena $\rightarrow$ Mamba
5. **Cơ chế Hyena** – long convolution + data-controlled gating + recurrence
6. **Hiện thực hiệu quả & kết quả paper** – matrix view, FFTConv, complexity
7. **Thực nghiệm** – Transformer-small và Hyena-small trên WikiText-2



1

Bối cảnh & Động lực

Vì sao attention $O(L^2)$ là nút thắt · long-context · capability gap

Phần 1



"Is attention all we need?"

- Self-attention là **trái tim** của Transformer và tạo nên hầu hết thành công của nó.
- Nhưng attention có **chi phí bậc hai** $O(L^2)$ theo độ dài chuỗi $L \rightarrow$ đặt **giới hạn cứng** lên lượng ngữ cảnh.
- Các toán tử dưới-bậc-hai trước đây (Lformer, Reformer, Performer, sparse...) đều phải **lai ghép** với attention dày đặc mới đạt chất lượng.

Câu hỏi trung tâm (paper · Section 1):

“Are there subquadratic operators that, inspired by its properties, are able to match its quality at scale?”

→ Mục tiêu: toán tử **không-attention**, rẻ hơn, **không cần hybridization**, vẫn sánh ngang Transformer.



- Nhiều bài toán thực tế cần **ngữ cảnh dài**: cả cuốn sách, mã nguồn dài, hội thoại dài, văn bản luật.
- Ngoài ngôn ngữ: **chuỗi sinh học (DNA / protein)**, âm thanh dài, ảnh gigapixel.
- Nhưng L tăng $\rightarrow$ attention tăng chi phí theo $L^2 \rightarrow$ nhanh chóng **hết bộ nhớ (OOM)** và **chậm**.

"Phá vỡ rào cản bậc hai" mở ra: dùng cả textbook làm ngữ cảnh, sinh nhạc dài, xử lý ảnh cực lớn ([paper](#) · [Section 1](#)).



2

Nhắc lại nền tảng

Language Modeling · Perplexity · kiến trúc Transformer

Phần 2



Autoregressive LM – dự đoán token kế tiếp:

$$P(w_1, \dots, w_n) = \prod_{t=1}^n P(w_t \mid w_{<t})$$

Perplexity – thước đo chất lượng LM (càng **thấp** càng tốt):

$$\text{PPL} = \exp\left(-\frac{1}{N} \sum_{t=1}^N \log P(w_t \mid w_{<t})\right) = \exp(\text{loss})$$

Trong PyTorch: $\text{PPL} = \exp(\text{validation cross-entropy})$. Đây cũng là metric nhóm dùng ở phần thực nghiệm (Quang).



Một Transformer Block (Pre-LN + residual):

$$\begin{aligned}\mathbf{x} &\leftarrow \mathbf{x} + \text{MHA}(\text{LN}(\mathbf{x})) \\ \mathbf{x} &\leftarrow \mathbf{x} + \text{FFN}(\text{LN}(\mathbf{x}))\end{aligned}$$

→ **Self-Attention** là phép **trộn thông tin** giữa các token – và cũng chính là **nút thắt** ta cần phân tích.

FFN xử lý **từng vị trí độc lập** (chi phí tuyến tính theo L) · **MHA** là phần **duy nhất** cho các token "nhìn nhau" – và cũng là nơi phát sinh chi phí $O(L^2)$.



3

Self-Attention & nút thắt

Cơ chế Q, K, V · ma trận attention · $O(L^2)$ · FlashAttention

Phần 3



$$\text{Attention}(Q, K, V) = \text{SoftMax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

- **B1.** Chiếu tuyến tính: $Q = XW_Q$, $K = XW_K$, $V = XW_V$.
- **B2.** Điểm tương quan: $S = QK^\top / \sqrt{d_k} \rightarrow$ ma trận $L \times L$.
- **B3.** Causal mask + SoftMax theo hàng $\rightarrow$ trọng số A .
- **B4.** Tổng có trọng số: $\text{Output} = AV$.

Multi-Head: chạy nhiều phép attention song song với các W_Q, W_K, W_V khác nhau, rồi nối lại.



- Mỗi token tính tương quan với **mọi token** → ma trận $L \times L$.
- **Causal LM**: token tại t chỉ "nhìn" $t' \leq t$ → mask **tam giác dưới** ($-\infty$ ở tương lai, sau SoftMax = 0).
- Số ô cần tính/lưu tỉ lệ với L^2 .

→ Đây là lý do attention **không mở rộng** được khi L rất lớn.

		k0	k1	k2	k3	
q0	[	●	·	·	·	]
q1	[	●	●	·	·	]
q2	[	●	●	●	·	]
q3	[	●	●	●	●	]

● attend (quá khứ) · · bị mask



L	Số ô L^2	L tăng 2×
256	65,536	—
512	262,144	~4×
1,024	1,048,576	~4×
8,192	67,108,864	~4×
64,000	$\sim 4.1 \times 10^9$	→ OOM

- **Thời gian:** $QK^\top$, SoftMax, AV đều $\sim O(L^2d)$ mỗi lớp.
- **Bộ nhớ:** lưu ma trận $L \times L$ cho backprop $\Rightarrow O(L^2)$.

L tăng **2×** → chi phí tăng **~4×**. Đây là rào cản chính cần phá vỡ.



- **FlashAttention** (Dao et al., 2022) tối ưu **truy cập bộ nhớ (IO-aware)**: không vật chất hóa toàn bộ ma trận $L \times L$, tính theo khối trên SRAM.
- → Giảm mạnh **bộ nhớ thực tế**, nhanh hơn 2–4× so với attention thường.

Nhưng: số phép tính vẫn là $O(L^2)$ – FlashAttention tối ưu **cách chạy**, không đổi **độ phức tạp**. Vẫn khó vượt tới ngưỡng cực dài.

→ Cần một toán tử có **độ phức tạp dưới bậc hai** thật sự, không chỉ tối ưu hằng số.



4

Khoảng cách năng lực & Related Work

Ba năng lực cốt lõi · SSM → H3/GSS → Hyena → Mamba

Phần 4



Vì sao các toán tử rẻ trước đây **thua** attention? Paper dùng **mechanistic interpretability** để rút ra **3 năng lực** cần giữ:

Năng lực	SSM / Conv tĩnh	Attention
Data control – phụ thuộc dữ liệu	Không (tĩnh)	Có – $A(u)$
Unrestricted context – ngữ cảnh không giới hạn	Không (bị locality)	Có
Sublinear params – tham số $\perp$ độ dài chuỗi	Có	Có

Hyena được thiết kế để **giữ đồng thời cả ba** – thay vì **xấp xỉ** attention, ta **tái dựng** tính chất của nó bằng primitive rẻ hơn.



Attention 2017



SSM



S4 2021



H3 / GSS 2022



Hyena 2023



Mamba '23

Kiến trúc	Phép trộn	Gating	Train	Ghi chú
Transformer	Attention	—	$O(L^2)$	đầy đủ ngữ cảnh, nghẽn L^2
S4	SSM (conv)	Không	$O(L \log L)$	học phụ thuộc dài, thiếu data-control
H3 / GSS	SSM (conv)	Có	$O(L \log L)$	attention-free đầu tiên sánh Transformer 125M
Hyena	Implicit long conv	Có (recurrence bậc N)	$O(N L \log L)$	tổng quát hóa H3/GSS , filter tự do
Mamba	Selective SSM	Có	$O(L)$	ra sau Hyena, filter động theo input

Hyena bậc 2 $\Leftrightarrow$ H3 · Hyena bậc 1 $\Leftrightarrow$ GSS. "Hierarchy" = họ toán tử có thứ bậc, mô hình cũ là tầng thấp.



5

Cơ chế Hyena

Long convolution · data-controlled gating · recurrence · hierarchy

Phần 5



Câu hỏi trọng tâm: Hyena thay thế self-attention bằng toán tử nào, và vì sao toán tử đó rẻ hơn khi context dài?

Attention làm tốt	Vấn đề khi L dài	Hyena thay bằng
Trộn thông tin toàn chuỗi	Ma trận $L \times L$	Long convolution
Trọng số phụ thuộc nội dung	So sánh mọi cặp token	Data-controlled gating
Tính toán trên context dài	$O(L^2)$ time/memory	FFTConv $O(L \log L)$

Điểm quan trọng: Hyena không tối ưu attention cũ, mà đề xuất một operator attention-free mới.



Hyena = Long Convolution + Data-Controlled Gating

Long convolution

- Filter dài gần bằng sequence.
- Mang tín hiệu từ token xa.
- Tính nhanh bằng FFTConv.

Gating

- Sinh từ projection của input.
- Nhân element-wise với output conv.
- Tạo tính content-aware.

Input u

-> projections: gates $x_1 \dots x_N$, value v

-> $z_1 = v$

-> repeat $n=1..N$: $z(n+1) = x(n) * \text{Conv}(h(n), z(n))$

-> output y

Câu nói ngắn: convolution truyền thông tin xa, gating quyết định giữ thông tin nào.



Từ local kernel sang full-context filter

CNN thường

Kernel ngắn, local
Nhìn vài token lân cận

Hyena

Filter dài bằng sequence
Nhận tín hiệu từ rất xa phía trước

$$(h * u)_t = \sum_{i=0}^t h_{t-i} u_i$$

CNN local: token t chỉ nhận từ vùng gần $[x \ x \ x] \text{ ---- } t$

Long conv: token t có thể nhận từ rất xa phía trước $[x \ x \ x \ x \ x \ x \ x \ x \ x] \ t$

Ý nghĩa: long convolution giúp mô hình hóa phụ thuộc xa mà không cần ma trận attention $L \times L$



Vì sao cần gating?

- Convolution thuần thường khá **tĩnh**: cùng filter áp dụng cho mọi input.
- Language modeling cần chọn lọc theo **ngữ cảnh input**.
- Hyena dùng gate x^n được chiếu từ input, rồi nhân với output convolution.

```
conv output:  
[0.8, 0.4, 0.9, 0.2]
```

```
gate x:  
[1.0, 0.1, 0.7, 0.0]
```

```
selected signal:  
[0.8, 0.04, 0.63, 0.0]
```

```
gate cao -> giữ  
gate thấp -> giảm
```

Long convolution đưa thông tin từ xa về; gating quyết định thông tin nào nên được giữ lại.

Công thức trung tâm

$$z_t^{n+1} = x_t^n \cdot (h^n * z^n)_t$$

Thành phần	Ý nghĩa
$z^1 = v$	value stream ban đầu
h^n	long filter ở bước n
x^n_t	gate tại token t
N	số bậc/order

Bước n :

1. lấy $z(n)$
2. Conv với filter $h(n)$
3. nhân gate $x(n)$
4. ra $z(n+1)$

Output cuối: $y = z^{(N+1)}$.



Vì sao gọi là "Hierarchy"?

- Hyena có thể lặp nhiều bước **convolution + gating**.
- **N** càng lớn, operator càng biểu diễn phong phú hơn.
- H3 có thể xem như Hyena bậc 2; GSS như Hyena bậc 1 với filter SSM.
- Trong repo nhóm: Hyena-small dùng **order = 2** để dễ reproduction.

Order 1: $z_1 \rightarrow \text{Conv } h_1 \rightarrow \text{Gate } x_1 \rightarrow z_2$

Order 2: $z_1 \rightarrow \text{Conv } h_1 \rightarrow \text{Gate } x_1 \rightarrow z_2 \rightarrow \text{Conv } h_2 \rightarrow \text{Gate } x_2 \rightarrow z_3$

Order N: repeat N lần $\rightarrow$ output



6

Hiện thực hiệu quả & Kết quả paper

Matrix view · implicit filter · FFTConv · complexity · kết quả paper

Phần 6



Trực giác ma trận

Thành phần Hyena	Dạng ma trận	Trực giác
Gating x^n	Diagonal matrix D_x	nhân từng vị trí/channel
Convolution $h^n * z^n$	Toeplitz matrix S_h	trượt cùng một filter qua chuỗi

Hyena xen kẽ D_x và S_h , thay vì tạo một attention matrix dense $L \times L$.

Attention: $y = A(x) \cdot v$ A là dense $L \times L$

Hyena: $y \approx D_{x2} \cdot S_{h2} \cdot D_{x1} \cdot S_{h1} \cdot v$

Điểm chính: vẫn phụ thuộc input qua D_x , nhưng tận dụng cấu trúc để tính rẻ hơn.

Filter dài nhưng ít tham số

$$h_t = \text{Window}(t) \cdot \text{FFN}(\text{PE}(t))$$

Cách học filter	Ý tưởng	Vấn đề/lợi ích
Explicit	lưu trực tiếp <code>h[0...L-1]</code>	dài hơn thì thêm tham số
Implicit	học hàm sinh <code>h_t</code> từ vị trí <code>t</code>	tham số nằm trong FFN

position `t` $\rightarrow$ Positional Encoding $\rightarrow$ small FFN $\rightarrow$ raw filter value
raw value $\ast$ Window(`t`) $\rightarrow$ `h_t`

Window/decay giúp filter ổn định hơn ở các khoảng cách xa.

Tính long convolution hiệu quả

Ý tưởng: chuyển convolution sang miền tần số để phép convolution trở thành phép nhân element-wise.

1. `FFT(filter)` và `FFT(signal)`
2. Nhân element-wise trong miền tần số
3. `iFFT` để quay lại sequence output

Direct convolution

Chi phí tăng nhanh khi filter rất dài
Ít phù hợp với long-context

FFTConv

Chi phí khoảng $O(L \log L)$
Có lợi khi L lớn



Điểm rơi của Hyena là context dài

Method	Compute theo L	Ý nghĩa thực tế
Standard Attention	$O(L^2)$	tạo ma trận $L \times L$
FlashAttention	$O(L^2)$	giảm bộ nhớ, tối ưu IO
Hyena	$O(N \cdot L \log L)$	không tạo attention matrix

Trong Hyena, N là order/số bước recurrence, thường được chọn nhỏ.

Khi L tăng rất lớn, $L \log L$ tăng chậm hơn L^2 , nên lợi thế của Hyena rõ nhất ở long-context.

Lưu ý: Hyena không nhất thiết nhanh hơn ở L nhỏ vì FFT có overhead.

Giá trị của paper: vừa có novelty kiến trúc, vừa thu hẹp quality gap với Transformer ở long-context.

Paper chứng minh 2 thứ: quality và efficiency

Quality

Benchmark	Kết quả
WikiText-103	Hyena-3 ~ Transformer 125M
The Pile 335M	Hyena-2 ~ GPT baseline
Associative recall	giữ chất lượng ở 30K-131K, nhiều baseline OOM

Efficiency

Length	Kết quả runtime
2K	gần crossover
8K	~5x vs attention, ~2x vs FlashAttention
64K	~100x vs FlashAttention

Kết quả chính của paper: **Hyena thu hẹp khoảng cách chất lượng với Transformer, đồng thời thể hiện lợi thế rõ ở các bài toán context rất dài.**



Thực nghiệm

WikiText-2 · Transformer-small vs Hyena-small · perplexity & tốc độ

Phần 7



Nhóm không tái hiện toàn bộ paper, vì The Pile 800GB và GPU lớn vượt quá tài nguyên môn học. Thay vào đó nhóm làm tái hiện ở quy mô nhỏ, mục tiêu là **kiểm chứng xu hướng** chứ không khớp số tuyệt đối.

Hai câu hỏi nhóm muốn tự trả lời:

1. Ở quy mô nhỏ, **perplexity** của Hyena có ngang Transformer không?
2. Khi **chuỗi dài** ra, Hyena có **lợi thế tốc độ** như paper dự đoán không?

Code do nhóm tự cài bằng thuần PyTorch, có đối chiếu với bản chính chủ [HazyResearch/safari](https://github.com/HazyResearch/safari).



Dữ liệu: WikiText-2, tokenizer GPT-2, cắt chuỗi theo độ dài 256.

Cấu hình	Transformer-small	Hyena-small
Layers	4	4
<code>d_model</code> / <code>d_ff</code>	256 / 1024	256 / 1024
Trộn thông tin	4 heads attention	order N=2, FFTConv
Params	16.1M	16.3M

Hyena dùng `torch.fft.rfft` thuần PyTorch, không có custom CUDA kernel.

Huấn luyện bằng AdamW kèm warmup rồi cosine, đánh giá bằng val loss quy ra perplexity, chạy trên Colab T4.



Công thức paper (Tiến vừa trình bày):

$$z_t^{n+1} = x_t^n \cdot (h^n * z^n)_t$$

Vòng lặp trong code của nhóm:

```
for n in range(self.order):  
    h_n = h_all[n]  
    x_n = gate_tensors[n]  
    conv = causal_fft_conv(h_n, z)  
    z = x_n * conv
```

Đối chiếu với bản chính chủ

HazyResearch/safari :

Thành phần lõi	Bản nhóm
FFTConv đệm 2L → cắt L đầu	khớp
Short conv depthwise	khớp
Số projection = N+1	khớp
Gating đệ quy bậc N	khớp
Activation của filter	SiLU (gốc: sin)
Modulation · skip-conn	rút gọn · bỏ

Nhóm không chạy lại code tác giả (cần CUDA kernel riêng, khó dựng trên Colab) mà cài lại phần lõi rồi đối chiếu.

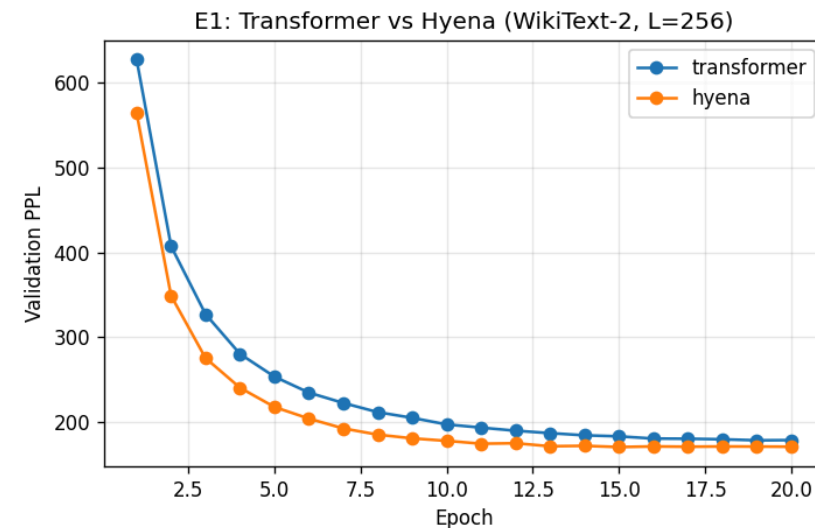
Kết quả E1: Perplexity (L=256)



Model	Val loss	Val PPL
Transformer-small	5.18	178.0
Hyena-small	5.14	170.1

Hai model cùng cỡ tham số cho perplexity gần như **ngang nhau** – Hyena nhỉnh hơn **~4%**, và thấp hơn ở **mọi epoch**.

20 epoch trên WikiText-2. Đường PPL đã đi ngang (gần hội tụ).

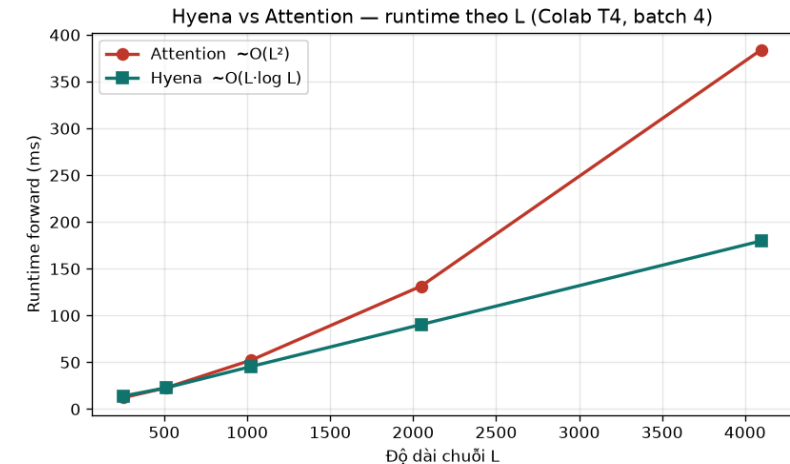


Val perplexity theo epoch

Đủ để kết luận: kiến trúc **không dùng attention** vẫn học ngôn ngữ ngang Transformer ở cùng cỡ tham số – Hyena là một baseline hợp lệ ở quy mô này.

Forward với input giả, ~16M tham số, batch 4, Colab T4:

L	TF (ms)	Hyena (ms)	Tỉ lệ
256	12.3	14.1	0.87×
512	22.8	22.7	1.00×
1024	52.3	45.5	1.15×
2048	131.4	90.4	1.45×
4096	383.9	179.7	2.14×



Runtime forward theo độ dài chuỗi

- **Tốc độ:** Hyena chậm hơn ở $L=256$ (overhead FFT), hòa quanh $L=512$, rồi vượt dần. Mỗi lần gấp đôi L : attention tăng $\sim \times 2.5-3$ (bình phương), Hyena chỉ $\times 2$ (tuyến tính) – throughput Hyena giữ **$\sim 90K$ token/s** ở mọi L , TF tụt từ 83K xuống 43K.
- **Bộ nhớ:** gần bằng nhau (3297 vs 3234 MB ở $L=4096$) – ở `d_model` nhỏ này khác biệt $O(L^2)$ chưa lộ rõ; lợi thế thấy được nằm ở **thời gian**.



- Ở chuỗi ngắn Hyena chậm hơn, vì chi phí FFT và đệm 2L lớn hơn phần tiết kiệm được. Hyena hòa ở khoảng $L = 512$, vượt lên từ $L = 1024$ và nhanh **2.14*** ở $L = 4096$, đúng như lưu ý trong paper.
- Bản cài của nhóm có **đơn giản hóa** so với safari (SiLU thay activation dạng sin, modulation rút gọn, bỏ skip-connection) – vì vậy hội tụ có thể chậm hơn bản gốc.
- Vì dùng thuần PyTorch và không có CUDA kernel, nhóm chưa đạt mức speedup tuyệt đối như paper.
- Phần đo tốc độ dùng input giả để đo thời gian forward, không phải đánh giá perplexity trên tập validation.
- Cần phân biệt rõ: số trên WikiText-103, The Pile và speedup từ 8K đến 64K là của **paper gốc**, còn số WikiText-2 ở đây là của **nhóm**.



- Nhóm tự cài Hyena bằng thuần PyTorch, phần lỗi khớp với bản chính chủ ở FFTConv đệm $2L$, short conv depthwise và gating đệ quy bậc N .
- Quan sát đúng xu hướng chính của bài: tốc độ Hyena tăng gần tuyến tính, attention tăng theo bình phương, hai đường giao nhau quanh $L = 512$ và Hyena vượt lên rõ từ $L = 1024$.
- Bài học rút ra: hiểu được vì sao attention nghẽn ở chuỗi dài, và thấy được giới hạn thực tế của tái hiện khi thiếu kernel tối ưu và tài nguyên.

Quy mô nhỏ nhưng đủ để thấy tận mắt cơ chế dưới bậc hai mà bài báo đề xuất.



Cảm ơn! · Q&A

Nhóm 08 · CS2308

Trần Tú Quang · Tô Huỳnh Minh Tiến · Nguyễn Cao Trung Kiên

Tài liệu: Poli et al., [Hyena Hierarchy](#), ICML 2023 · docs/poli23a.pdf

Tài liệu nhóm: README.md · notebooks/colab_E1_run.ipynb